런타임 진입점과 대화 엔진

CLI 프로세스가 어떻게 한 번의 요청을 지속 가능한 에이전트 실행으로 바꾸는지 코드의 책임 경계를 따라 읽는다

이 권을 마치면 main.tsx와 QueryEngine, query.ts를 “큰 파일 세 개”가 아니라 서로 다른 수명과 실패를 소유하는 세 층으로 설명할 수 있다.

STUDY SNAPSHOT
Claude Code v2.1.88
2ca5ddabfed5

제3자 복원 소스 기반
한국어 · Guided Edition

오늘 우리는 이 코드의 어느 부분을 함께 보는가

화면에서 보이는 프롬프트보다 먼저 프로세스가 실행 형태를 선택하고, QueryEngine이 세션 수명을 붙잡으며, query.ts가 모델 응답과 도구 결과가 교대하는 한 turn의 프로토콜을 수행한다. 우리는 이 셋을 한 줄짜리 흐름으로 축약하지 않고 각각이 보존해야 하는 상태와 실패 의미를 확인한다.

50	33,709	243	357
분류된 TS/TSX 파일	물리 코드 줄	async 함수	tool use 표기

CHAPTER 01

프로세스는 먼저 실행 형태를 고른다

같은 바이너리가 대화형 터미널, 비대화형 출력, SDK 제어, 특수 진입점을 품으려면 초기화와 제품 동작을 분리해야 한다.

CHAPTER 02

QueryEngine은 세션의 수명을 붙잡는다

QueryEngine의 핵심은 API를 호출하는 데 있지 않다. 여러 turn을 가로질러 메시지, 취소, 사용량, 권한 거부와 파일 관찰 상태를 일관되게 유지하는 데 있다.

CHAPTER 03

query.ts는 한 turn의 프로토콜을 수행한다

모델 응답과 도구 실행이 반복된다는 설명만으로는 부족하다. 각 반복에서 메시지 pairing, budget, recovery, stop 이유가 닫혀야 다음 반복이 유효해진다.

CHAPTER 04

Provider 경계에서 의미를 보존한다

서비스 API 코드는 HTTP 호출 래퍼가 아니다. 내부 메시지와 모델별 요청 형식, cache, retry, fallback의 차이를 흡수하는 anti-corruption layer다.

분포 수치는 중요도 점수가 아니다. 이 권이 직접 펼쳐보는 코드 표면의 규모와 언어 메커니즘 밀도를 가늠하기 위한 탐색 표지다.

01

프로세스는 먼저 실행 형태를 고른다

같은 바이너리가 대화형 터미널, 비대화형 출력, SDK 제어, 특수 진입점을 품으려면 초기화와 제품 동작을 분리해야 한다.

이 장에서 실제로 펼쳐볼 경로

src/entrypoints/cli.tsx · src/main.tsx · src/bootstrap/** · src/replLauncher.tsx

entrypoint는 단순한 main 함수가 아니다

인자 하나가 렌더러·입출력 계약·오류 표면을 바꾼다.

같이 `src/entrypoints/cli.tsx`를 펼치면 가장 먼저 보이는 것은 거대한 기능 구현이 아니라 실행 전 조건을 정리하는 얇은 경계다. 이 경계는 환경과 인자를 읽고 실제 CLI 구현을 불러오며, 번들 진입점이 제품 내부 모듈에 직접 달라붙지 않게 한다.

이 얇음은 사소하지 않다. 번들러가 요구하는 시작점, 런타임이 요구하는 전역 초기화, 애플리케이션이 요구하는 모드 선택을 한 파일에 섞으면 테스트와 대체 진입점이 모두 어려워진다. entrypoint는 "무엇을 할지"보다 "어떤 세계에서 시작하는지"를 소유한다.

변경을 검토할 때는 새 옵션을 어느 파일에 추가했는지보다 그 옵션이 프로세스 초기화에 속하는지, 세션 구성에 속하는지, turn 동작에 속하는지를 먼저 묻자. 이 질문이 `main.tsx`의 추가 비대화를 막는 첫 번째 기준이다.

스터디에서 확인할 질문

새 실행 모드를 넣는다면 어떤 책임까지 entrypoint에 두고 어디서부터 composition으로 넘길 것인가?

main.tsx의 크기는 우연이 아니라 결합 신호다

4천 줄이 넘는 파일은 부트스트랩, 설정, 인증, 모드 선택이 한 교차점에 모였음을 보여 준다.

`main.tsx`는 환경 검사, 설정 적재, 인증, 명령 처리, interactive/print 분기, 화면 구성으로 이어지는 여러 수명을 만난다. 그래서 이 파일을 "앱 시작 코드"라고만 부르면 왜 작은 수정이 넓은 회귀 위험을 갖는지 설명하지 못한다.

여기서 중요한 것은 줄 수 자체가 아니라 서로 다른 변경 이유가 같은 영역에 있다는 점이다. 인증 정책 변경, CLI 옵션 추가, REPL 렌더링 변경은 각각 다른 이해관계인데 같은 composition root에서 부딪힌다. 큰 파일은 곧 나쁜 파일이 아니지만, 변경 이유의 수를 세게 만드는 관찰점이다.

리팩터링은 함수를 임의로 잘라내는 일이 아니다. 먼저 순수한 옵션 해석, 외부 상태를 읽는 초기화, UI를 만드는 composition을 구분하고, 각 단계 사이에 전달되는 값을 명시해야 한다. 그래야 "작아졌지만 숨은 전역 상태는 그대로"인 분해를 피할 수 있다.

스터디에서 확인할 질문

main.tsx 안에서 독립적으로 검증 가능한 첫 번째 seam은 어디이며, 그 seam이 전달해야 할 최소 상태는 무엇인가?

대화형과 비대화형은 출력 형식만 다르지 않다

입력을 기다리는 주체와 결과를 소비하는 주체가 달라지면 런타임 계약도 달라진다.

대화형 경로에서는 Ink 화면이 사용자 입력, 권한 질문, 진행 상태, 중단을 함께 다룬다. 반면 print/headless 경로에서는 호출자가 메시지 스트림과 종료 상태를 소비한다. 같은 QueryEngine을 사용해도 누가 `next()`를 호출하고 누가 permission decision을 제공하는지가 달라진다.

이 차이는 UI를 떼어낸 정도가 아니다. 대화형에서는 사람의 응답을 기다릴 수 있지만 headless에서는 자동 거부나 외부 host callback이 필요하다. 터미널 상태를 전제로 한 로직이 engine 내부로 새면 SDK는 곧바로 불완전한 두 번째 구현이 된다.

새 기능을 추가할 때 "REPL에서 보인다"와 "engine 결과에 존재한다"를 구분하자. 먼저 canonical event와 상태 전이를 만들고, REPL과 SDK가 각각 그것을 표현하도록 해야 두 표면이 같은 의미를 유지한다.

스터디에서 확인할 질문

권한 요청처럼 대화형 대기가 필요한 사건을 headless 소비자에게는 어떤 타입의 이벤트로 노출해야 하는가?

초기화 순서는 숨은 의존성 지도다

환경·설정·인증·도구·프롬프트는 서로의 전제이므로 순서가 곧 설계 정보다.

부트스트랩 코드를 읽을 때 함수 이름만 나열하지 말고 각 단계가 무엇을 생산하고 다음 단계가 무엇을 전제하는지 적어 보자. 예를 들어 cwd와 설정이 정해져야 permission context를 만들 수 있고, 도구 집합과 모델이 정해져야 system prompt를 구성할 수 있다.

이렇게 읽으면 초기화 순서가 단순한 procedural boilerplate가 아니라 dependency graph의 직렬화라는 점이 보인다. 전역 singleton이나 환경 변수를 뒤늦게 읽는 함수가 있다면 그래프 밖의 숨은 edge이며, 재현성과 테스트를 약화시킨다.

구조를 개선하려면 모든 것을 DI 컨테이너로 옮길 필요는 없다. 단계별 산출물을 작은 immutable configuration으로 만들고, 늦게 필요한 외부 자원은 명시적 factory로 남기는 편이 이 코드의 현재 형태에 더 맞다.

스터디에서 확인할 질문

초기화 중간에 실패했을 때 이미 만들어진 자원의 정리 책임은 어느 단계가 가져야 하는가?

함께 펼쳐볼 자료 src/bootstrap/state.ts · src/main.tsx:520-760 · src/replLauncher.tsx

02

QueryEngine은 세션의 수명을 붙잡는다

QueryEngine의 핵심은 API를 호출하는 데 있지 않다. 여러 turn을 가로질러 메시지, 취소, 사용량, 권한 거부와 파일 관찰 상태를 일관되게 유지하는 데 있다.

이 장에서 실제로 펼쳐볼 경로

src/QueryEngine.ts · src/utils/abortController.ts · src/utils/processUserInput/**

필드 목록에서 세션의 불변조건을 읽는다

클래스의 private state는 이 객체가 대신 기억해 주는 약속의 목록이다.

`mutableMessages`, `abortController`, 누적 usage, permission denials, read-file state는 서로 다른 이유로 turn 이후에도 남는다. 메시지는 다음 모델 호출의 과거이고, read-file state는 편집의 전제이며, 거부 목록은 SDK 결과의 설명 책임이다.

이 필드들을 한데 묶은 이유는 모두 "한 번의 submit 호출"보다 긴 수명을 갖기 때문이다. 반대로 turn 안에서만 필요한 임시 상태가 여기에 쌓이기 시작하면 engine은 세션 owner와 작업 scratchpad의 경계를 잃는다.

새 필드를 추가할 때는 먼저 수명을 적자: process, engine/session, turn, tool call 중 어디까지 살아야 하는가. 이 질문 하나로 reset 위치, 직렬화 여부, 동시 접근 위험이 결정된다.

스터디에서 확인할 질문

discoveredSkillNames가 매 submit마다 clear되는 이유와 readFileState가 유지되는 이유를 수명 관점에서 비교해 보라.

submitMessage는 문자열을 받는 함수가 아니라 turn을 여는 문이다

한 호출 안에서 프롬프트 구성, 입력 정규화, query 실행, 결과 축적이 결합된다.

메서드 초반의 구조 분해는 engine이 의존하는 capability를 드러낸다. 도구, MCP client, budget, model, app-state 접근자, permission callback이 모두 이번 turn의 실행 환경을 만든다. 이 목록은 API 옵션 모음이 아니라 turn이 성립하기 위한 포트다.

이후 system prompt parts를 모으고 user input context를 만들며, query가 내보내는 사건을 SDK 메시지로 투영한다. 중요한 점은 request payload가 한 번에 만들어지고 끝나는 것이 아니라 입력 처리와 동적 기능 발견 때문에 turn 중에도 context가 보강될 수 있다는 것이다.

이 메서드를 분리한다면 "줄 수" 기준이 아니라 setup, protocol execution, projection, settlement의 경계로 잘라야 한다. 각 단계가 실패했을 때 어느 상태까지 durable한지 함께 정의하지 않으면 분해 후에도 의미는 더 흐려진다.

스터디에서 확인할 질문

submitMessage가 중간에 abort될 때 호출자에게 반드시 남겨야 하는 최소 결과는 무엇인가?

async generator는 turn의 시간을 타입에 넣는다

결과 하나가 아니라 부분 관찰의 연속을 반환하기 때문에 consumer가 실행 속도와 취소를 함께 제어한다.

`async *submitMessage`는 Promise 하나로는 표현하기 어려운 중간 메시지, tool progress, 최종 결과를 차례로 내보낸다. `await`는 내부 비동기 작업을 기다리고 `yield`는 외부 소비자에게 현재까지의 관찰을 넘긴다.

따라서 generator는 단순한 스트리밍 문법이 아니다. producer의 실행이 consumer의 `next()`에 맞춰 재개되고, 취소·오류·조기 return이 protocol 종료 의미를 갖는다. 영상에서 강조된 generator의 장점은 바로 이 지점에서 프로젝트의 실제 구조와 만난다.

다만 generator가 내구성을 제공하지는 않는다. yield된 사건과 transcript에 기록된 사건이 어긋날 수 있으므로, 관찰 스트림과 durable journal의 책임을 따로 확인해야 한다.

AsyncGenerator가 표현하는 producer ↔ consumer 계약

분석용 TypeScript — 실제 구현의 제어 구조를 축약·재작성

```
async function* submit(input): AsyncGenerator<Event> {
  const ctx = await prepare(input)
  for await (const event of runTurn(ctx)) {
    yield project(event)
  }
  await settle(ctx)
}
```

1. **await**
   producer 내부의 비동기 작업이 끝날 때까지 현재 재개 지점을 보존한다.
2. **yield**
   완료가 아니라 현재 관찰 하나를 consumer에게 넘기고 suspended-yield 상태가 된다.
3. **return/throw**
   iterator 종료와 실패도 protocol의 일부이며 외부 effect 정리는 별도다.

언어·런타임 안쪽에서는 async generator 객체는 상태와 resume 요청 큐를 보유한다. Promise 하나처럼 한 번 완료되는 값이 아니라 next() 호출마다 진행되는 상태 기계다.

스터디에서 확인할 질문

consumer가 generator를 더 이상 순회하지 않을 때 실행 중인 provider stream과 tool effect는 어떻게 닫혀야 하는가?

AbortSignal은 취소 요청이지 자동 롤백이 아니다

하나의 signal이 여러 비동기 경계를 통과해도 이미 일어난 외부 효과는 되돌아가지 않는다.

engine이 소유한 AbortController는 provider 요청과 tool context가 공통의 취소 의도를 보게 한다. 각 하위 작업은 signal을 관찰하고 가능한 지점에서 멈춰야 한다. 이 협력적 모델 덕분에 강제 스레드 중단 없이도 정리 코드를 실행할 수 있다.

하지만 사용자가 중단을 누른 순간 파일 write가 이미 끝났다면 signal은 그 효과를 취소하지 못한다. 따라서 UI의 "중단됨", query protocol의 종료, 실제 파일 시스템 상태를 같은 것으로 취급하면 안 된다.

취소 설계는 세 질문으로 검토하자. 새 작업 시작을 막는가, 진행 중 I/O에 signal을 전달하는가, 부분 효과 뒤 어떤 결과를 journal에 남기는가. 이 셋이 모여야 사용자가 본 상태와 실제 상태가 다시 합쳐진다.

취소 요청이 실제 중단으로 번역되는 위치

01	→	02	→	03	→	04
UI/Host		**Controller**		**Provider/Tool**		**Settlement**
사용자가 interrupt를 요청한다.		AbortController가 signal을 한 번 abort한다.		각 작업이 signal을 관찰하고 새 I/O를 멈춘다.		이미 발생한 효과와 중단 결과를 기록한다.

이 그림이 감추지 않는 것 AbortSignal은 broadcast되는 상태와 사건일 뿐 thread를 강제 종료하지 않는다. listener와 I/O API가 협력해야 하며 completed write는 취소되지 않는다.

스터디에서 확인할 질문

Bash process와 파일 편집은 같은 AbortSignal을 받아도 왜 서로 다른 settlement가 필요한가?

03

query.ts는 한 turn의 프로토콜을 수행한다

모델 응답과 도구 실행이 반복된다는 설명만으로는 부족하다. 각 반복에서 메시지 pairing, budget, recovery, stop 이유가 닫혀야 다음 반복이 유효해진다.

이 장에서 실제로 펼쳐볼 경로

src/query.ts · src/query/config.ts · src/query/stopHooks.ts · src/services/tools/**

루프보다 먼저 protocol state를 찾는다

query.ts를 while문으로 요약하면 가장 중요한 예외 상태가 사라진다.

이 파일은 provider 호출, streaming message, tool-use 수집, tool-result 생성, compact와 retry를 한 turn 안에서 조율한다. 반복 자체보다 중요한 것은 현재 메시지가 누구의 응답인지, tool_use id가 어떤 result를 기다리는지, stop_reason이 무엇을 허용하는지다.

이를 state machine으로 읽으면 조건문이 "잡다한 예외"가 아니라 허용된 전이의 guard로 보인다. 예를 들어 tool result가 빠진 상태로 다음 provider 호출을 하면 API protocol 자체가 깨질 수 있다.

스터디에서는 각 branch를 실행 순서로 외우지 말고 state와 invariant로 적어 보자. 그러면 파일을 분해할 때도 함수 경계가 아니라 상태 소유권을 기준으로 제안할 수 있다.

한 turn의 허용 상태



`tool_use` MODEL → TOOLS

`tool_result` TOOLS → MODEL 또는 SETTLE

`abort/error` 현재 effect 관찰 후 SETTLE

`stop_reason` MODEL → SETTLE → DONE

언어·런타임 안쪽에서는 while문은 상태 기계의 한 구현일 뿐이다. 실제 안전성은 상태별 허용 event와 tool id pairing, terminal transition에 있다.

스터디에서 확인할 질문

현재 turn을 가장 작은 상태 집합으로 표현한다면 model phase, tool phase, settlement phase에 어떤 필드가 필요한가?

streaming은 텍스트 조각 이상의 사건을 운반한다

모델 출력은 content delta, usage, stop, error가 섞인 typed event stream이다.

API stream을 소비하는 코드는 텍스트를 이어 붙이는 동시에 assistant message의 정체성과 usage를 갱신하고, tool-use block이 완성되는 시점을 판별한다. 즉 화면에 보이는 토큰과 실행에 필요한 구조화 블록이 같은 transport에서 다른 소비자를 가진다.

이 구조에서 "첫 토큰 latency"와 "메시지 완결성"은 서로 다른 품질이다. UI는 부분 텍스트를 빨리 보여 줄 수 있지만 tool executor는 arguments가 닫힐 때까지 기다려야 한다. 한 스트림을 여러 projection으로 읽는 이유다.

새 event를 추가한다면 query 내부 처리, SDK projection, transcript persistence, REPL 표시 중 어떤 소비자가 반드시 알아야 하는지 명시해야 한다. 그렇지 않으면 한 표면만 조용히 의미를 잃는다.

스터디에서 확인할 질문

부분 assistant message를 저장하거나 노출할 때 재개 시 중복과 불완전 블록을 어떻게 구별할 것인가?

tool_use와 tool_result는 하나의 대화 원자다

외부 효과는 두 메시지 사이에서 일어나지만 다음 모델 호출에는 짝이 맞는 기록으로 들어가야 한다.

모델이 tool_use id를 내보내면 runtime은 해당 도구를 찾고 실행한 뒤 같은 id의 tool_result를 user-side content로 만든다. 성공뿐 아니라 unknown tool, permission denial, abort도 result 형태로 돌아가야 protocol이 닫힌다.

이 pairing은 데이터베이스 transaction과 동일하지 않다. 실제 shell이나 파일 효과는 이미 일어났을 수 있고 원자적 롤백도 없다. 여기서 원자성은 대화 protocol 관점의 관찰 완결성에 가깝다.

따라서 오류 처리는 예외를 던지는 것으로 끝나지 않는다. 다음 모델이 무엇을 보게 되는지, transcript가 어떤 상태로 재개되는지까지 포함해야 한다. tool id는 효과·관찰·복구를 잇는 correlation key다.

스터디에서 확인할 질문

도구 구현이 throw한 경우에도 왜 tool_result 형태의 관찰을 남겨야 하는가?

budget과 stop은 정책이 protocol을 자르는 방식이다

max turns, 비용, token budget은 숫자 옵션이 아니라 종료 의미를 만든다.

query는 모델의 자연 종료만 기다리지 않는다. 호출 횟수와 비용, 사용자 취소, hook 결과, provider 오류가 실행을 끝낼 수 있다. 각 종료 원인은 사용자와 SDK가 다른 후속 행동을 선택할 수 있도록 구분되어야 한다.

budget check가 루프 바깥에 한 번만 있으면 직전 호출의 사용량을 반영하지 못하고, tool 실행 뒤에만 있으면 이미 새 효과가 시작될 수 있다. 어디에서 검사하는지가 정책의 실제 의미다.

새 제한을 넣을 때는 측정 단위, 검사 시점, 초과 전 허용 오차, 이미 시작된 작업 처리, 결과 표현을 함께 설계하자. 숫자 상수 하나만 추가하는 변경은 거의 항상 불완전하다.

스터디에서 확인할 질문

비용 상한에 도달했을 때 현재 tool_use를 실행해야 하는가, 거부 result를 만들어야 하는가, turn 전체를 중단해야 하는가?

04

Provider 경계에서 의미를 보존한다

서비스 API 코드는 HTTP 호출 래퍼가 아니다. 내부 메시지와 모델별 요청 형식, cache, retry, fallback의 차이를 흡수하는 anti-corruption layer다.

이 장에서 실제로 펼쳐볼 경로

src/services/api/claude.ts · src/utils/queryContext.ts · src/upstreamproxy/**

내부 메시지와 provider payload를 같은 타입으로 두지 않는 이유

세션이 기억해야 할 사실과 특정 API가 요구하는 직렬화는 수명이 다르다.

내부에는 UI용 메타데이터, transcript 식별자, attachment provenance가 있지만 provider는 정해진 role/content/tool schema만 받는다. API 경계는 이 차이를 지우는 곳이 아니라 의도적으로 변환하는 곳이다.

내부 타입을 provider SDK 타입으로 곧장 사용하면 SDK 업데이트가 세션 저장 형식까지 흔든다. 반대로 변환 함수가 너무 느슨하면 중요한 tool id나 cache marker가 소실된다. 경계의 품질은 필드 수가 아니라 보존해야 할 invariant의 명시성으로 평가해야 한다.

provider를 추가하거나 버전을 올릴 때는 request shape 비교뿐 아니라 stop reason, usage, streaming error가 내부 event로 어떻게 정규화되는지 확인하자.

스터디에서 확인할 질문

provider가 새로운 content block을 추가했는데 내부 union이 모르면 그것을 버릴 것인가, 보존할 것인가, 명시적으로 실패할 것인가?

prompt cache는 성능 기능이면서 표현 순서의 제약이다

재사용 가능한 prefix를 만들려면 system·tool·message 배열의 안정성이 필요하다.

Prompt caching은 같은 앞부분을 다시 보내는 비용과 latency를 줄이지만, cache breakpoint 앞의 작은 변화가 큰 miss를 만들 수 있다. 따라서 context builder의 순서와 동적 정보 배치가 곧 캐시 전략이 된다.

이 프로젝트에서 cache control과 prompt 구성은 별도 관심사가 아니다. tools나 system context가 매 turn 불필요하게 흔들리면 기능상 정답이어도 운영 비용은 악화된다. 반대로 cache hit을 위해 오래된 정보를 고정하면 정확성이 깨진다.

성능 개선은 hit rate만 보지 말고 어떤 정보가 얼마나 자주 변하며, 그 정보를 prefix 뒤로 옮겨도 의미가 같은지 검토해야 한다. 표현의 안정성과 최신성 사이의 tradeoff다.

스터디에서 확인할 질문

시간·cwd·동적 MCP instructions 중 어떤 정보를 cacheable prefix에 둘 수 있고, 어떤 것은 매 turn 갱신해야 하는가?

retry와 fallback은 같은 요청의 반복이 아닐 수 있다

stream 중간 실패와 호출 전 실패는 이미 관찰된 출력과 비용이 다르다.

연결 전에 실패하면 같은 request를 다시 보내는 것이 비교적 단순하지만, 일부 토큰이나 tool_use가 이미 전달된 뒤라면 재시도는 중복 메시지와 중복 효과 가능성을 만든다. fallback model은 여기에 모델 행동 차이까지 추가한다.

따라서 retry 정책은 오류 코드 목록보다 commit point를 먼저 정의해야 한다. consumer에게 어느 사건이 보였는지, assistant message가 journal에 들어갔는지, tool execution이 시작됐는지가 재시도 가능성을 결정한다.

오류 처리 코드를 리팩터링할 때 모든 provider error를 공통 helper로 합치는 유혹을 경계하자. 공통화할 것은 분류와 backoff이고, 부분 관찰 뒤의 settlement는 protocol layer가 소유해야 한다.

재시도 가능성을 가르는 commit point

실패 시점	이미 관찰된 것	단순 retry 위험
요청 전	없음	상대적으로 낮음
첫 delta 전	요청 비용 가능	중복 billing 확인 필요
partial text 후	UI/SDK가 일부 출력 소비	중복 assistant content
tool_use 후	외부 effect 시작 가능	중복 실행

구분해야 하는 이유 retry 판단은 error code만으로 결정되지 않는다. consumer-visible event와 effect 시작 여부가 시도 identity를 바꾼다.

스터디에서 확인할 질문

첫 content delta 이후 연결이 끊겼을 때 "같은 turn 재시도"가 안전하다는 것을 어떤 상태로 판단할 수 있는가?

엔진을 바꿀 때 보존해야 할 것은 호출 순서가 아니라 의미다

리팩터링의 oracle은 사용자에게 보이는 최종 문자열보다 넓다.

QueryEngine과 query.ts를 분해하려면 메시지 pairing, permission denial, cancellation, usage aggregation, transcript resume를 동등성 기준으로 잡아야 한다. 이 중 하나라도 잃으면 happy path 테스트가 통과해도 runtime 의미는 달라진다.

좋은 seam 후보는 context preparation, model stream adapter, tool phase, turn settlement다. 각각 입력과 출력이 typed value로 닫히며 실패 책임도 다르다. 반면 “helpers” 폴더로 조건문을 이동하는 것은 소유권을 만들지 않는다.

작은 단계로 옮길 때 기존 함수와 새 함수가 같은 event trace를 만드는지 shadow comparison을 사용할 수 있다. 단, 실제 tool effect를 두 번 실행하지 않도록 provider와 effect port를 기록/재생 가능한 경계로 바꿔야 한다.

스터디에서 확인할 질문

리팩터링 전후를 비교하는 최소 trace에는 어떤 event와 상태 snapshot이 들어가야 하는가?

코드 뒤에서 작동하는 언어·런타임 메커니즘

문법 이름을 외우기보다, 런타임이 실제로 무엇을 저장하고 언제 제어를 넘기는지와 이 프로젝트가 그것을 어떤 책임에 사용하는지를 함께 본다.

async generator

언어가 제공하는 것
Promise로 감싼 iterator 요청 큐와 suspended/executing/completed 상태를 유지한다.

이 저장소가 쓰는 방식
QueryEngine과 query가 부분 메시지·도구 진행·최종 결과를 시간 순서로 내보낸다.

놓치기 쉬운 함정
yield는 저장이나 rollback이 아니다. consumer가 멈췄을 때 외부 효과 정리가 별도로 필요하다.

for await…of

언어가 제공하는 것
각 next() 결과를 await하고 iterator가 끝날 때까지 consumer 쪽 제어 흐름을 반복한다.

이 저장소가 쓰는 방식
provider stream과 tool update를 같은 소비 문법으로 연결한다.

놓치기 쉬운 함정
느린 consumer가 backpressure를 만들지만 내부 별도 queue가 있으면 메모리는 계속 늘 수 있다.

AbortController

언어가 제공하는 것
signal에 한 번 abort 사건을 발행하는 협력적 취소 primitive다.

이 저장소가 쓰는 방식
세션의 중단 의도를 provider 요청과 tool context에 전파한다.

놓치기 쉬운 함정
이미 완료된 파일·process 효과는 되돌리지 않는다.

class private state

언어가 제공하는 것
instance 생애 동안 mutable reference와 누적 상태를 보존한다.

이 저장소가 쓰는 방식
메시지, usage, denial, read-file cache를 여러 turn에 걸쳐 소유한다.

놓치기 쉬운 함정
turn 임시값까지 instance에 넣으면 reset 경계와 동시성 의미가 흐려진다.

structural typing

언어가 제공하는 것
명목 선언보다 필요한 property shape의 호환성으로 값을 연결한다.

이 저장소가 쓰는 방식
config port와 event projection을 구현체에 덜 결합시킨다.

놓치기 쉬운 함정
넓은 object와 cast는 서로 다른 identity를 우연히 호환시킬 수 있다.

dynamic import / adapter

언어가 제공하는 것
module loading 시점과 dependency availability를 runtime까지 늦춘다.

이 저장소가 쓰는 방식
provider·feature별 경계를 core가 직접 고정하지 않게 한다.

놓치기 쉬운 함정
실패 시점이 늦어지고 누락 모듈이 compile-time 검증에서 빠질 수 있다.

스터디용 코드 탐색 인덱스

파일을 처음부터 끝까지 읽지 않는다. 아래 시작점에서 symbol을 잡고 정의·호출·상태·실패 경로를 왕복한다.

시작 경로	먼저 볼 표현	그 다음에 물을 질문
src/entrypoints/cli.tsx	진입 함수와 import 경계	process 초기화와 application composition이 어디서 갈리는가
src/main.tsx	mode 분기와 dependency 생성	서로 다른 변경 이유가 어느 상태를 공유하는가
src/QueryEngine.ts	private fields와 submitMessage	session/turn/tool-call 수명이 어떻게 구분되는가
src/query.ts	async generator와 stop branch	허용 상태 전이와 pairing invariant는 무엇인가
src/query/tokenBudget.ts	검사 함수와 호출 시점	제한 초과 전에 이미 시작된 효과는 무엇인가
src/services/api/claude.ts	request lowering과 stream adapter	provider 세부가 내부 message에 새는가
src/utils/queryContext.ts	ordered transformation	어떤 metadata가 사라지고 어떤 id가 남는가
src/utils/abortController.ts	signal 생성·결합	취소 요청 뒤 settlement owner는 누구인가

코드 독해 규칙

타입 선언만 보고 runtime 검증을 가정하지 않는다. 호출 순서만 보고 authority를 가정하지 않는다. UI 표시만 보고 durable state를 가정하지 않는다. 주석만 보고 production 활성이나 upstream 의도를 확정하지 않는다.

한 번의 turn을 코드에서 재구성해 보기

순서를 베끼지 말고, 각 단계가 어떤 상태를 소유하며 무엇을 다음 단계에 보장하는지 기록한다.

1. `entrypoints/cli.tsx`에서 선택된 실행 모드가 `main.tsx`의 어느 composition으로 이어지는지 찾는다.
2. `QueryEngine.submitMessage`의 config 구조 분해에서 session-level port와 turn-level option을 두 색으로 구분한다.
3. `query.ts`에서 model phase와 tool phase를 나누고, 각 경계를 넘는 message id와 tool_use id를 적는다.
4. abort, permission denial, unknown tool, provider stream failure 네 반례가 어떤 SDK message와 transcript 상태를 남기는지 비교한다.
5. 리팩터링 seam 하나를 고르고 입력·출력·실패·동등성 oracle을 한 장에 작성한다.

> **완료의 기준**
>
> 한 turn을 "사용자 → 모델 → 도구 → 모델" 화살표가 아니라 상태·식별자·종료 의미를 가진 protocol로 설명할 수 있으면 된다.

다음 모임에 남길 기록

다음 모임에는 가장 위험한 숨은 전역 의존성 하나와, 그것을 명시적 입력으로 바꿀 때 생길 비용을 가져온다.